

Información general del manual del proyecto.....	2
Instalación del proyecto.....	2
Python.....	2
Entorno virtual.....	2
GitHub.....	2
Clonado.....	3
Postman.....	3
Configuración del entorno y herramientas.....	3
Dependencias.....	3
.env.....	3
Stripe.....	3
Stripe CLI.....	4
Uso del entorno.....	4
Base de datos.....	4
Proveedor.....	4
Transacción.....	5
Incidencia.....	5
Ejecución.....	5
Webhook.....	5
Servidor.....	5
Endpoints principales.....	5
API.....	5
Create-payment.....	5
Refund-payment.....	6
Webhook.....	6

Información general del manual del proyecto

Las siguientes explicaciones contendrán algunos comandos que pueden variar según el sistema operativo, para este caso se explicara con comandos de windows.

A lo largo del archivo habrán distintos enlaces a páginas de descargas, documentos y páginas oficiales para facilitar la instalación y uso del proyecto, estos enlaces están marcados por texto azul y subrayado.

Instalación del proyecto

Python

Para descargar python, se debe acceder a su [página de descarga oficial](#) y seleccionar la versión deseada de este.

Tras descargar el ejecutable e iniciarlo es importante seleccionar las casillas "Use admin privileges..." y "Add Python.exe to PATH" que se encuentran situadas en la parte inferior de la ventana de instalación, esto nos permitira usar python mas comodamente desde cualquier terminal.

Tras esto podemos asegurarnos que python este descargado con el comando:

```
python --version
```

Entorno virtual

A la hora de instalar el proyecto suele ser recomendable la creación de un entorno virtual para evitar conflictos entre las dependencias.

Para crear un entorno virtual se debe usar el comando:

```
python -m venv .NOMBRE_ENTORNO
```

Tras esto, es necesario activarlo con el siguiente comando:

```
.NOMBRE_ENTORNO\Scripts\activate.bat
```

GitHub

El primer paso consiste en ir a GitHub y descargar el [código del proyecto](#), este se tiene que extraer en el directorio que se desee, ya que es un proyecto público en GitHub no es necesario contar con una cuenta para poder descargarlo mediante el botón de código,

situado en la parte central derecha de la parte superior de los archivos que componen el proyecto, pero es recomendable contar con una cuenta para la gestión de versiones.

Clonado

Para realizar el clonado se puede hacer de diversas formas, a continuación voy a explicar el caso de github desktop ya que así se puede usar para el control de versiones.

Tras descargar el ejecutable desde su [página oficial](#) e instalarlo siguiendo las indicaciones del ejecutable, hay que clonar el repositorio desde la opción de “add”, dentro de esta opción seleccionaremos la opción de url y pegaremos la url que se encuentra dentro de la opción “code” del repositorio y seleccionaremos la ruta donde deseamos clonar este repositorio

Postman

Para descargar postman hay que dirigirse a su [página oficial](#) y descargar el ejecutable, esto nos permitirá llamar al servidor de forma externa simulando llamadas desde un entorno cliente

Configuración del entorno y herramientas

Dependencias

Para un correcto funcionamiento es necesario descargarse las dependencias del archivo “requirements.txt”, situado en la carpeta docs, con el siguiente comando:

pip install -r docs/requirements.txt

Si durante el proceso de instalación se ha decidido modificar la ruta del archivo requirements.txt se deberá cambiar la última parte del comando a la ruta actual de este.

.env

El siguiente paso consiste en crear un archivo .env el cual no se incluye en el repositorio de GitHub gracias al gitignore, ya que este contiene datos sensibles que no se deben hacer publicos.

Dentro del archivo .env hay que crear tres variables, STRIPE_PUBLIC_KEY, STRIPE_SECRET_KEY y STRIPE_WEBHOOK_SECRET que se rellenaran con datos que obtendremos en los siguientes pasos.

Stripe

El siguiente paso consiste en contar con una cuenta de Stripe.

Desde Stripe podremos obtener el valor de las variables STRIPE_PUBLIC_KEY y STRIPE_SECRET_KEY, que se encuentran entre las opciones de los desarrolladores del entorno de Stripe que se piensa usar.

Stripe CLI

Para asignarle el valor a la variable STRIPE_WEBHOOK_SECRET se deben seguir unos pasos más. Se debe instalar Stripe CLI, siguiendo sus instrucciones para windows, se debe ir [a su GitHub](#) y descargar la versión correspondiente del sistema operativo y arquitectura adecuados.

Tras descargar y descomprimir el archivo se obtendrá un ejecutable, en la barra de ruta donde se encuentre el ejecutable hay que cambiar toda la ruta por “cmd”, lo cual abrirá una terminal en la ruta del ejecutable lo que nos permitirá usar este ejecutable.

En esta nueva terminal, hay que usar el siguiente comando:

stripe listen --forward-to localhost:8000/webhook/

Tras introducir el comando ocurrirán 2 cosas:

- Obtendremos un mensaje indicándonos que está funcionando y está escuchando el puerto 8000 del localhost (que es el que se usa en este proyecto), lo que nos permitirá realizar algún cambio que depende del funcionamiento de Stripe (como es el caso de las transacciones, al crearse se guardan como “pending”, al completarse los pagos se actualizan a “completed”).
- Obtendremos un código que empieza por whsec_, ese es el valor que hay que asignarle a STRIPE_WEBHOOK_SECRET

Postman

Con el fin de llevar una buena configuración de postman se va a suponer el servidor del entorno sera “127.0.0.1:8000” que tendrá que sustituir por la IP correspondiente.

Hecho esto hay que crear 5 requests esenciales

Get providers

Para generar este request se debe darle a la opción de más en las colecciones y seleccionar “Add request” asignándole un nombre apropiado para poder identificarlo más fácilmente.

En el apartado URL introduciremos la url: **http://127.0.0.1:8000/api/provider/**

Esto nos permitirá obtener los datos de todos los proveedores registrados en la base datos

Get payments

Para generar este request se debe darle a la opción de más en las colecciones y seleccionar “Add request” asignándole un nombre apropiado para poder identificarlo más fácilmente.

En el apartado URL introduciremos la url: **http://127.0.0.1:8000/api/payment/**

Esto nos permitirá obtener los datos de todos los pagos registrados en la base datos

Create provider

Para generar este request se debe darle a la opción de más en las colecciones y seleccionar "Add request" asignándole un nombre apropiado para poder identificarlo más fácilmente.

En el apartado URL introduciremos la url: **http://127.0.0.1:8000/api/provider/**

En la opción desplegable a la izquierda de la url seleccionaremos la opción POST
Esto nos permitirá registrar proveedores en la base datos

Create payment

Para generar este request se debe darle a la opción de más en las colecciones y seleccionar "Add request" asignándole un nombre apropiado para poder identificarlo más fácilmente.

En el apartado URL introduciremos la url: **http://127.0.0.1:8000/api/payments/**

En la opción desplegable a la izquierda de la url seleccionaremos la opción POST
Esto nos permitirá registrar transacciones en la base datos

Refun

Para generar este request se debe darle a la opción de más en las colecciones y seleccionar "Add request" asignándole un nombre apropiado para poder identificarlo más fácilmente.

En el apartado URL introduciremos la url: **http://127.0.0.1:8000/refund-payment/**

En la opción desplegable a la izquierda de la url seleccionaremos la opción POST
Esto nos permitirá registrar devoluciones (incidencias) en la base datos y realizar un cambio en la transacción correspondiente

Uso del entorno

El proyecto cuenta con una documentación automática mediante swagger, por lo que antes de realizar modificaciones en el proyecto es recomendable acceder a la ruta "127.0.0.1:8000/swagger/" y leer la documentación.

El entorno está diseñado para ser un backend por lo que no cuenta con interfaz gráfica más allá de la que cuenta swagger.

Base de datos

La base de datos es una base de datos SQLITE generada automáticamente con el proyecto Django.

La base de datos se compone de tres tablas, una por cada modelo usado en este proyecto.

Proveedor

Esta tabla está orientada a mantener un registro de que proveedores de servicio se usan y registrar si están en desarrollo, en pruebas o si están deshabilitados.

Transacción

Esta tabla está orientada al almacenamiento de las distintas transacciones realizadas, indicando con qué proveedor se han realizado y si se encuentran completas, pendientes de pago o devueltas.

Incidencia

Esta tabla se centra en llevar un registro de las incidencias en las transacciones (contando también con las devoluciones).

Al iniciar el servidor debe de contener unos datos usados tanto de pruebas como de ejemplo de las tablas de la base de datos, es conveniente el asegurarse de que hayan datos con los que trabajar o en su defecto crearlos.

Ejecución

Webhook

Para ejecutar el servidor correctamente es necesario realizar el paso de instalar Stripe CLI y ejecutar la terminal desde esa carpeta para poder trabajar adecuadamente con el ejecutable.

Tras usar el comando:

stripe listen --forward-to localhost:8000/webhook/

El token se reinicia y es importante actualizarlo en el archivo .env

Servidor

Para ejecutar el servidor solo hay que realizar el siguiente comando:

python manage.py runserver

Este hará que Django ejecute el servidor en localhost:8000

Postman

Get providers

Al hacer click en el botón “send” situado al lado derecho de la url postman enviará una petición al servidor para obtener los proveedores registrados en la base de datos

Los resultados se mostrarán en formato JSON en la parte central inferior de la ventana

Get payments

Al hacer click en el botón “send” situado al lado derecho de la url postman enviará una petición al servidor para obtener las transacciones registradas en la base de datos

Los resultados se mostrarán en formato JSON en la parte central inferior de la ventana

Create provider

Para poder hacer uso del POST es necesario enviarle datos al servidor en formato JSON, para ello se debe seleccionar la opción de “Body” situada debajo de la url y en el primer desplegable seleccionar la opción “raw”, lo que nos permitirá introducir datos en la parte central de la ventana, de forma similar a este ejemplo:

```
{  
  "name": "ejemplo",  
  "environment": "test",  
  "active": true  
}
```

Al hacer click en el botón “send” situado al lado derecho de la url postman enviará una petición al servidor para obtener los proveedores registrados en la base de datos

Los resultados se mostrarán en formato JSON en la parte central inferior de la pantalla

Create payment

Para poder hacer uso del POST es necesario enviarle datos al servidor en formato JSON, para ello se debe seleccionar la opción de “Body” situada debajo de la url y en el primer desplegable seleccionar la opción “raw”, lo que nos permitirá introducir datos en la parte central de la ventana, de forma similar a este ejemplo:

```
{  
  "provider_id": 1,  
  "amount": 10,  
  "currency": "USD"  
}
```

Al hacer click en el botón “send” situado al lado derecho de la url postman enviará una petición al servidor para obtener los proveedores registrados en la base de datos

La aplicación está preparada para aceptar euros como divisa predeterminada si no se añade el campo currency, este se puede cambiar según las indicaciones marcadas por la [documentación de stripe](#)

Los resultados se mostrarán en formato JSON en la parte central inferior de la pantalla

Refun

Para poder hacer uso del POST es necesario enviarle datos al servidor en formato JSON, para ello se debe seleccionar la opción de “Body” situada debajo de la url y en el primer desplegable seleccionar la opción “raw”, lo que nos permitirá introducir datos en la parte central de la ventana, de forma similar a este ejemplo:

```
{  
  "transaction_id": 20  
}
```

Al hacer click en el botón “send” situado al lado derecho de la url postman enviará una petición al servidor para obtener los proveedores registrados en la base de datos

Los resultados se mostrarán en formato JSON en la parte central inferior de la pantalla

Endpoints principales

API

Este endpoint es simplemente un CRUD de los distintos modelos.

Create-payment

Este endpoint crea una nueva transacción.

Para ello recibe el id de un proveedor y la cantidad del pago, también se le puede añadir el tipo de moneda según las indicaciones mostradas en la [documentación de Stripe](#), pero el endpoint está preparado para aplicar la divisa de euros por defecto en caso de no recibir este dato.

El endpoint devuelve un enlace y el id de la transacción, en el enlace permite procesar el pago y tras esto cambia el estado de pendiente a completado.

Este endpoint redirige al usuario a la ventana “127.0.0.1:8000/api/transaction/” para comprobar las transacciones realizadas, esto se puede modificar a voluntad.

Refund-payment

Este endpoint modifica una transacción para que sea una devolución y se cree automáticamente una incidencia registrada como devolución.

Para ello recibe el id de la transacción, con el id identifica la transacción y comprueba que el estado sea completado, asegurando que el pago se haya realizado y luego procede a hacer una devolución del dinero.

Webhook

Este endpoint está diseñado para poder realizar la actualización de pendiente a completada de las transacciones al verificar el pago de stripe.

Solo funciona si se han seguido los pasos indicados en el apartado de “Configuración del entorno y herramientas” y de “Ejecución”.